

Debreceni SZC Mechwart András Gépipari és Informatikai Technikum



Szoftverfejlesztő és -tesztelő

Vizsgaremek

Galaxy Portál

2026.

Készítette: Ónadi Viktor
Sipos Anita
Terdik Ferenc

- 1.BEVEZETÉS 3
 - 1.1. Témaválasztás indoklása 3
 - 1.2. Kitűzött célok 4
- 2.CSAPATMUNKA / A FEJLESZTÉS MENETE 5
- 3.FELHASZNÁLÓI DOKUMENTÁCIÓ 6
 - 3.1. Az alkalmazás telepítése 7
 - 3.2. Az alkalmazás indítása 8
 - 3.3. Az alkalmazás használata 9
- 4.FEJLESZTŐI DOKUMENTÁCIÓ 19
 - 4.1. Fejlesztői környezet és technológiák 19
 - 4.2. Adatbázis modell 23
 - 4.3. Kiemelt kódrészletek 27
 - 4.4. Tesztelés és hibajavítás 31
- 5.ÖSSZEGZÉS 28
- 6.TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK 29
- 7.IRODALOMJEGYZÉK 30

1.1. Témaválasztás indoklása

A Galaxy Portal egy csillagászati és űrkutatási témájú webes portál, amelyet vizsgaremekként készítettem el. A projekt ötlete abból a felismerésből született, hogy az űrkutatás és a csillagászat iránti érdeklődés az utóbbi években jelentősen megnövekedett, ugyanakkor kevés olyan magyar nyelvű, interaktív felület létezik, amely egyszerűen böngészhetővé tenné ezeket a tartalmakat.

A rendszer célja egy olyan webes alkalmazás létrehozása volt, amely lehetővé teszi a felhasználók számára égitestek, csillagászati események és szakcikk megtekintését, valamint eseményekre történő jelentkezést és kommentek írását. A portál kétféle felhasználói réteget szolgál ki: a vendég felhasználók a nyilvános tartalmakat (főoldal, cikkek) érhetik el, míg a regisztrált felhasználók a védett tartalmakat (égitestek, események, kommentek, profil) is használhatják.

A projekt során egy modern, korszerű technológiákat alkalmazó webalkalmazás létrehozása volt a cél, amely bemutatja a frontend fejlesztés területén szerzett felkészültséget, különös tekintettel a React keretrendszer használatára, az API-kommunikációra, a hitelesítési mechanizmusokra és a reszponzív felülettervezésre.

A fejlesztés során az alábbi célokat tűztem ki:

Funkcionális célok:

Égitestek listájának és részletes adatainak megjelenítése kártyás nézetben és modál ablakban
Csillagászati események listázása, részletes megtekintése, eseményre jelentkezés és leiratkozás
Csillagászati cikkek böngészése és olvasása
Komment rendszer működtetése (csak bejelentkezett felhasználóknak)
Felhasználókezelés: regisztráció, bejelentkezés, profil megtekintése
Adminisztrációs funkciók (felhasználók kezelése)

Technikai célok:

Modern, komponensalapú frontend React + Vite használatával
Robusztus backend Django + Django REST Framework használatával
JWT alapú hitelesítés és tokenfrissítés
Reszponzív, mobilbarát felület
Biztonságos API-kommunikáció
Karbantartható, jól dokumentált kódbázis

A rendszer kialakításakor törekedtem a későbbi bővíthetőség és továbbfejleszthetőség biztosítására.

2. A Fejlesztés menete

A projektet egyéni fejlesztés keretében valósítottam meg. A munka során az alábbi eszközöket és módszereket alkalmaztam.

Verziókezelés: A projekt forráskódját Git verziókezelővel és GitHub platformmal kezeltem. Ez lehetővé tette a kód változásainak nyomon követését, a különböző fejlesztési fázisok elkülönítését és a biztonságos visszaállítási pontok létrehozását.

Fejlesztési módszertan: A fejlesztés során iteratív megközelítést alkalmaztam. Az egyes funkciókat egymás után, modulonként fejlesztettem ki:

- 1. Backend alapok: Django projekt inicializálása, modellek kialakítása, REST API végpontok létrehozása.**
- 2. Frontend alapok: React alkalmazás inicializálása Vite-tal, projektmappa-struktúra kialakítása.**
- 3. Útvonalkezelés: React Router beállítása, védett útvonalak implementálása.**
- 4. Hitelesítési rendszer: JWT autentikáció a backendben, AuthContext a frontenden.**
- 5. API kommunikáció: Axios konfiguráció, interceptorok, API függvények.**
- 6. Tartalom oldalak: Égitestek, események, cikkek oldalak és részletes nézeteik.**
- 7. Interakciós funkciók: Eseményre jelentkezés, komment rendszer.**
- 8. Felhasználói felület: Stílusok, reszponzivitás, UI finomhangolás.**

Kommunikáció: A fejlesztés során a frontend és a backend közötti API kommunikációt folyamatosan egyeztettem és teszteltem.

3.1. Az alkalmazás telepítése

A Galaxy Portal futtatásához az alábbi szoftverek telepítése szükséges:

- **Python (3.8 vagy újabb):** <https://www.python.org/downloads/>
- **Node.js (18.x vagy újabb):** <https://nodejs.org/en/download>
- **Git:** <https://git-scm.com/install>

3.1.1. A projekt klónozása

cmd

```
git clone https://github.com/[felhasznalonev]/galaxy-portal-frontend.git  
cd galaxy-portal-frontend
```

3.1.2. Backend telepítése

```
# 1. Virtuális környezet létrehozása
py -m venv vizsga_env

# 2. Virtuális környezet aktiválása (Windows)
vizsga_env\Scripts\activate

# 3. Belépés a backend könyvtárba
cd backend

# 4. Függőségek telepítése
pip install -r requirements.txt

# 5. Adatbázis migrációk futtatása
py manage.py migrate

# 6. Superuser létrehozása (opcionális)
py manage.py createsuperuser
```

3.1.3. Frontend telepítése

```
# 1. Belépés a frontend könyvtárba
cd frontend

# 2. Függőségek telepítése
npm install
```

3.2. Az alkalmazás indítása

```
# 1. Virtuális környezet aktiválása (ha még nem
aktív)vizsga_env\Scripts\activate

# 2. Belépés a backend könyvtárba
cd backend

# 3. Szerver indítás
py manage.py runserver
```

A backend elérhető: <http://127.0.0.1:8000/>

3.2.2. Frontend szerver indítása

1. Belépés a frontend könyvtárba

```
cd frontend
```

2. Fejlesztői szerver indítása

```
npm run dev
```

A frontend elérhető: <http://localhost:5173/>

3.2.3. További indítások

Backend újraindítása:

- `cd backend`
- `py manage.py runserver`

Frontend újraindítása:

- `cd frontend`
- `npm run dev`

3.3.1. Regisztráció és bejelentkezés

Az alkalmazás a regisztráció és az azt követő bejelentkezés után válik teljes mértékben használhatóvá.

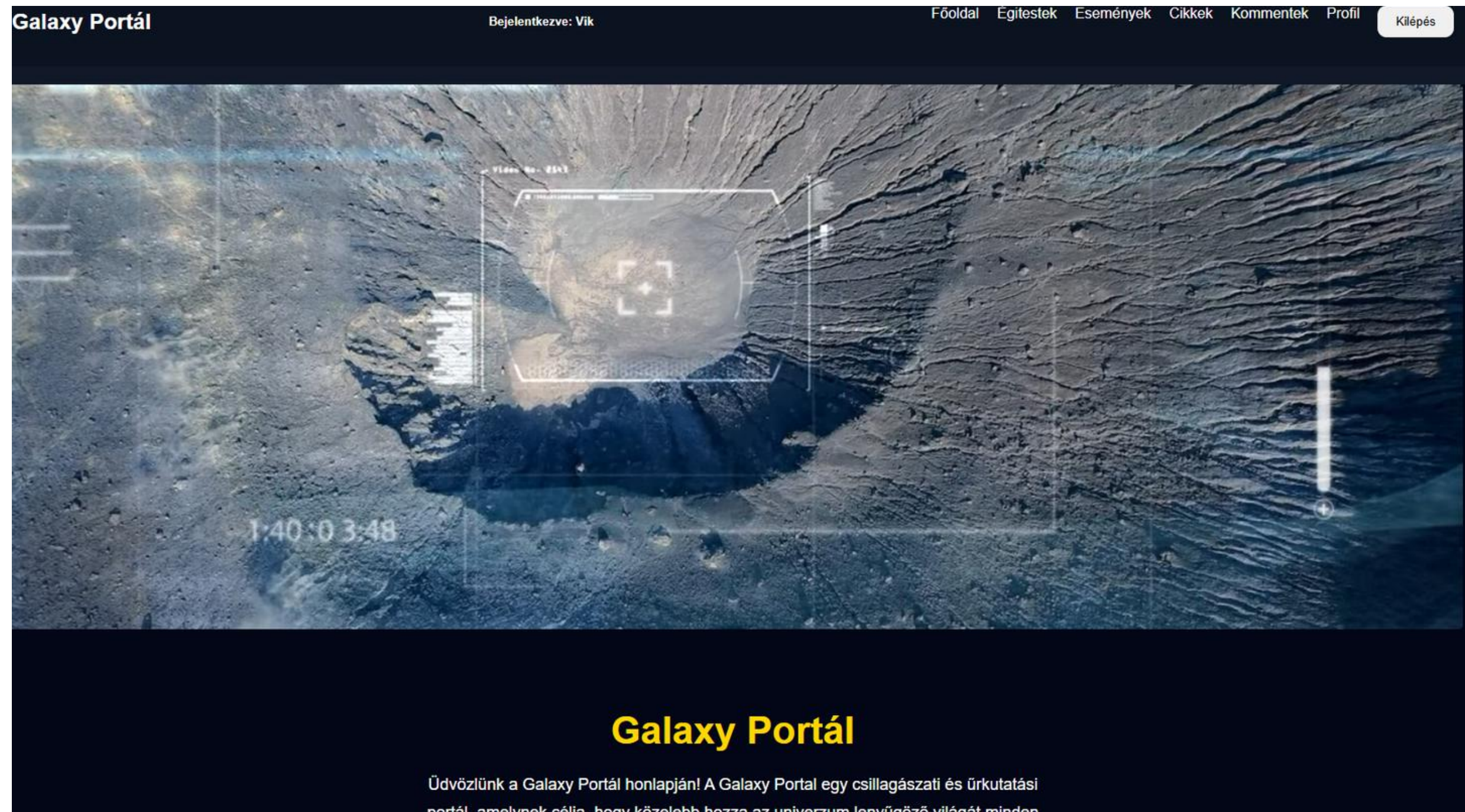
Regisztráció:

- Kattints a navigációs sáv "Regisztráció" gombjára
- Töltsd ki az űrlapot:
- Felhasználónév – válassz egy egyedi felhasználónevet
- E-mail cím – add meg az e-mail címedet
- Jelszó – válassz egy biztonságos jelszót (legalább 8 karakter)
- Jelszó megerősítése – add meg újra a jelszót
- Kattints a "Regisztráció" gombra
- Sikeres regisztráció esetén automatikusan bejelentkezel

Bejelentkezés:

- Kattints a navigációs sáv "Belépés" gombjára
- Add meg a felhasználóneved és jelszavad
- Kattints a "Bejelentkezés" gombra
- Sikeres bejelentkezés után a főoldalra kerülsz

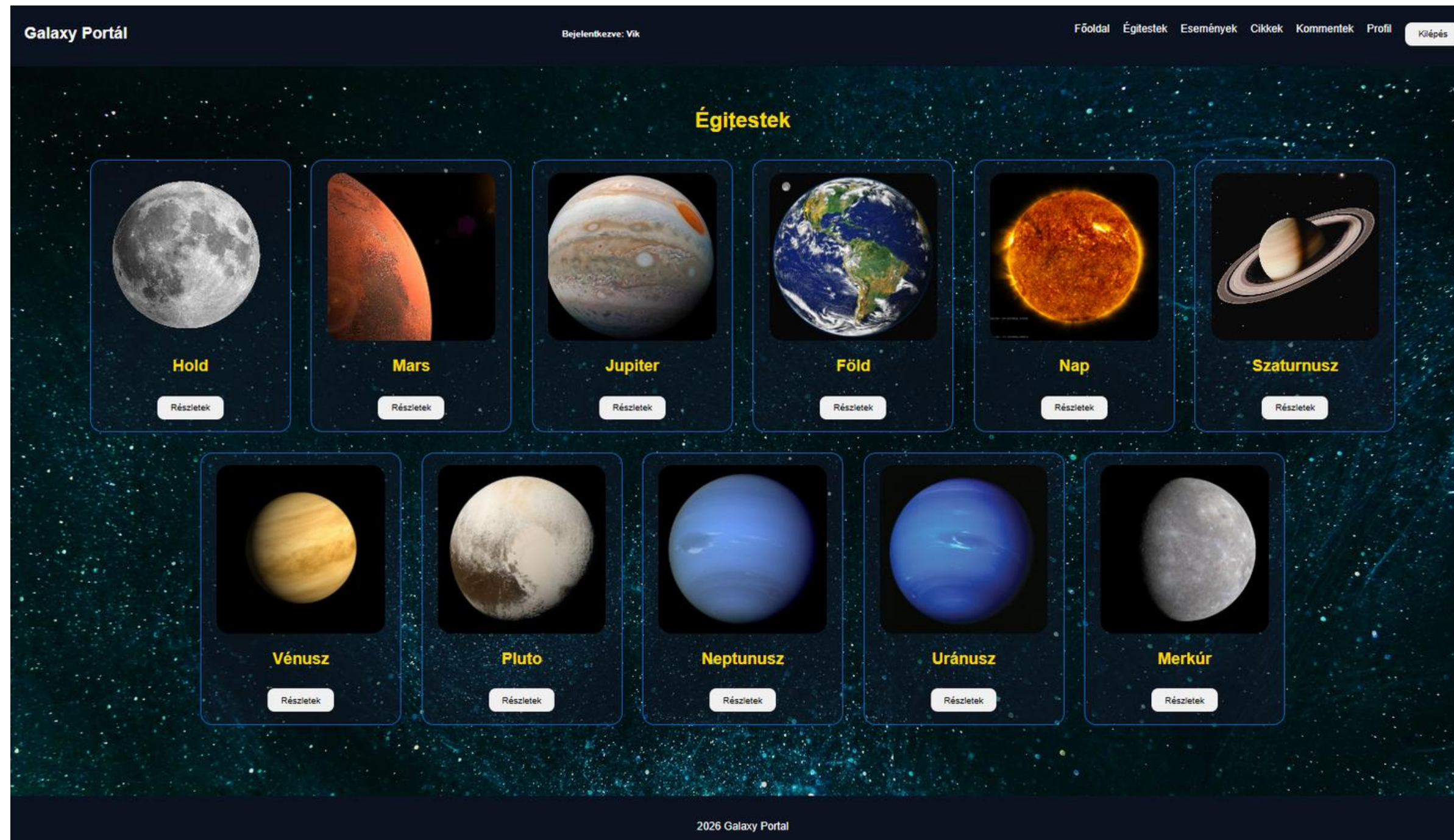
A főoldal egy űr-tematikájú videóval és bemutatkozó szöveggel fogadja a látogatókat. A navigációs sáv segítségével elérhetők a különböző oldalak. Az ablak átméretezése után reszponzív mobil nézetben a navigáció hamburger menüvé alakul.



Az Égitestek oldalon a rendszerben tárolt égitestek listája jelenik meg kártyás nézetben.

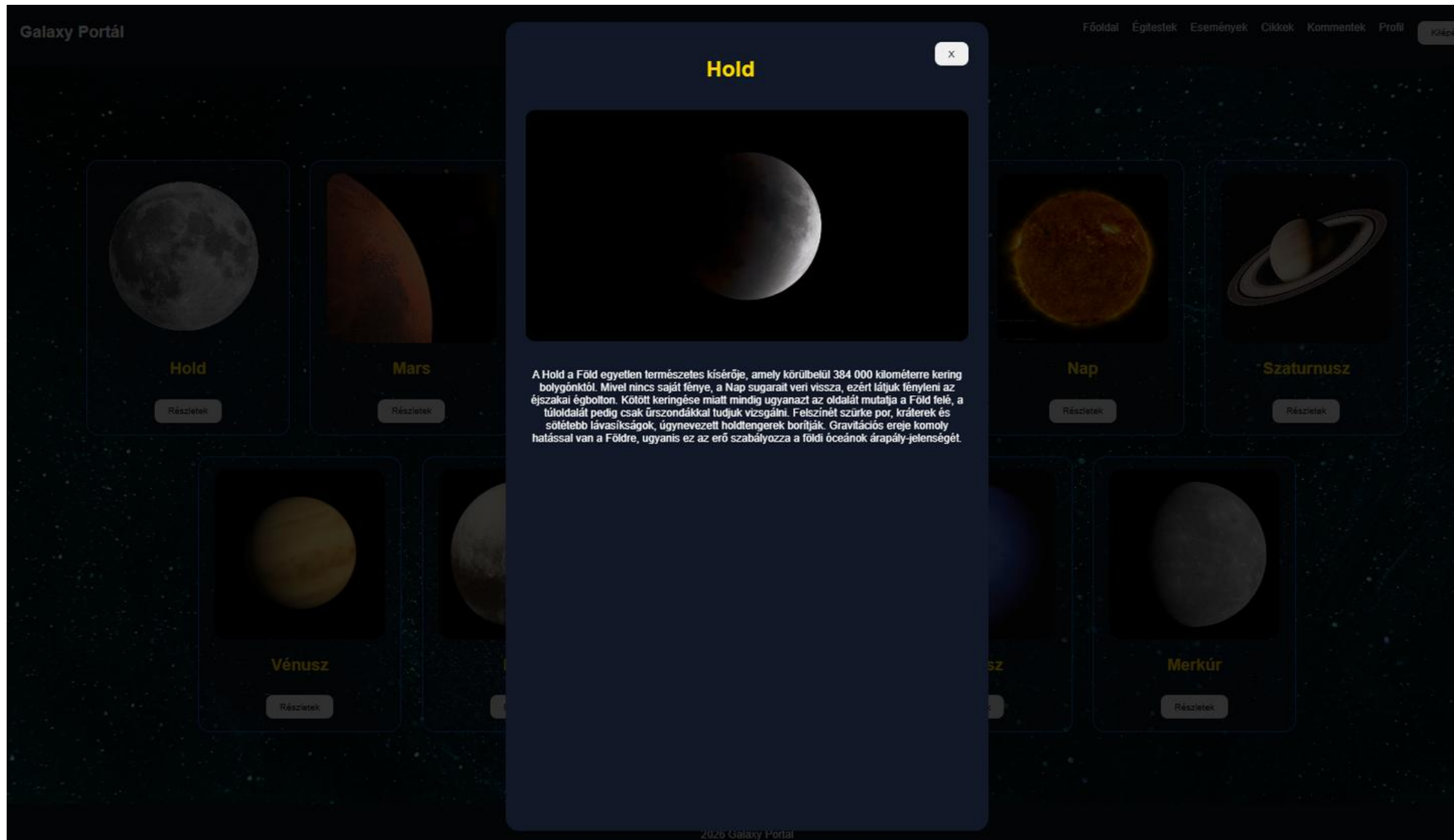
Égitestek böngészése:

1. Kattints a navigációs sáv "Égitestek" menüpontjára
2. Az égitestek képpel és névvel ellátott kártyákon jelennek meg



Részletes nézet megtekintése:

1. Kattints egy égitest "Részletek" gombjára
2. Egy modál(modális) ablak nyílik, amely tartalmazza:
 - Az égitest nevét
 - Egy WebM videót az égitestről
 - Az égitest részletes leírását
3. A modál bezárásához kattints az "X" gombra vagy a modálon kívülre



3.3.4. Események

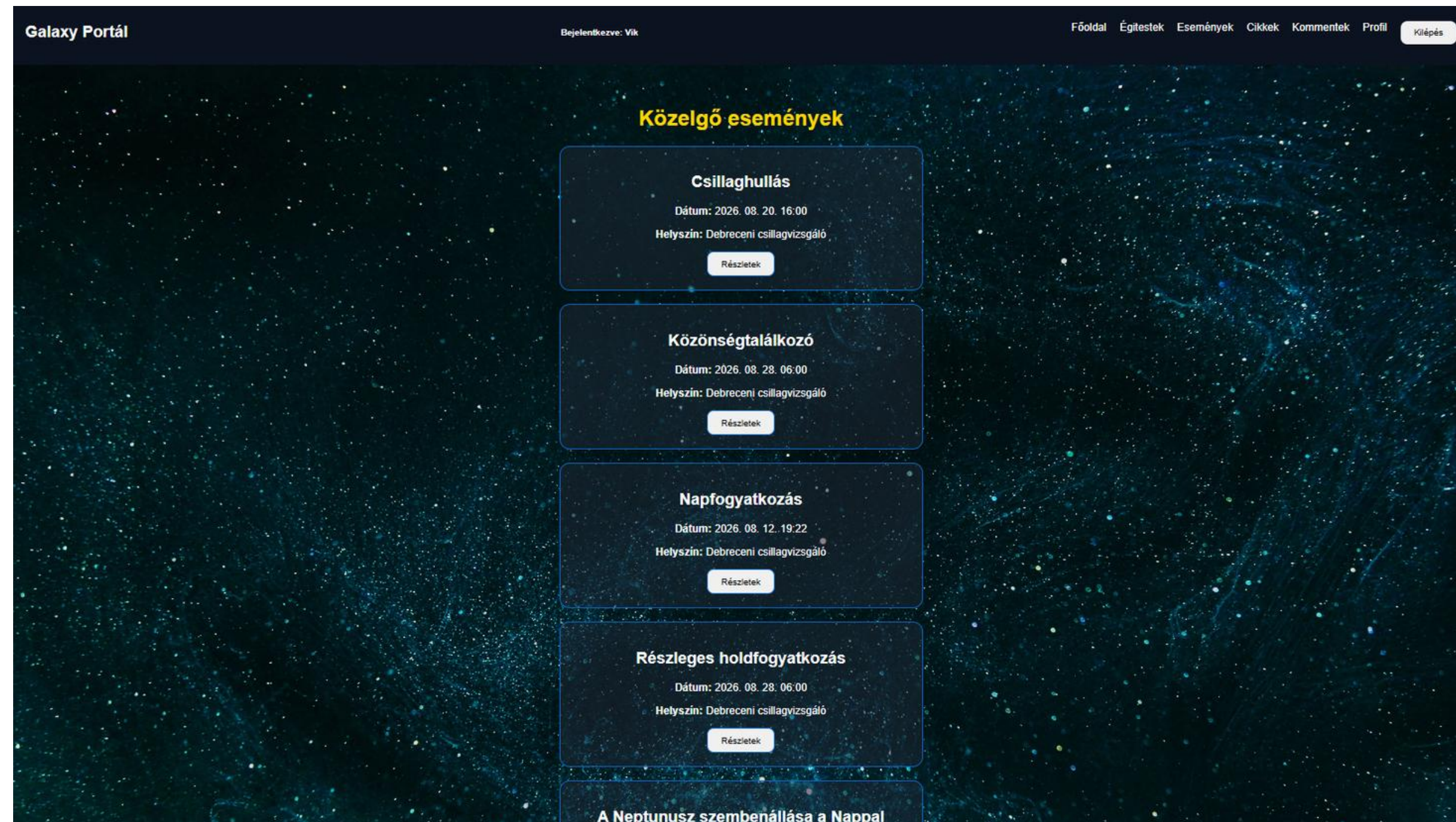
Az Események oldalon a csillagászati események listája jelenik meg kártyás nézetben.

Események böngészése:

1. Kattints a navigációs sáv "Események" menüpontjára
2. Minden kártyán látható:
 - Az esemény címe
 - Az esemény dátuma és időpontja
 - Az esemény helyszíne

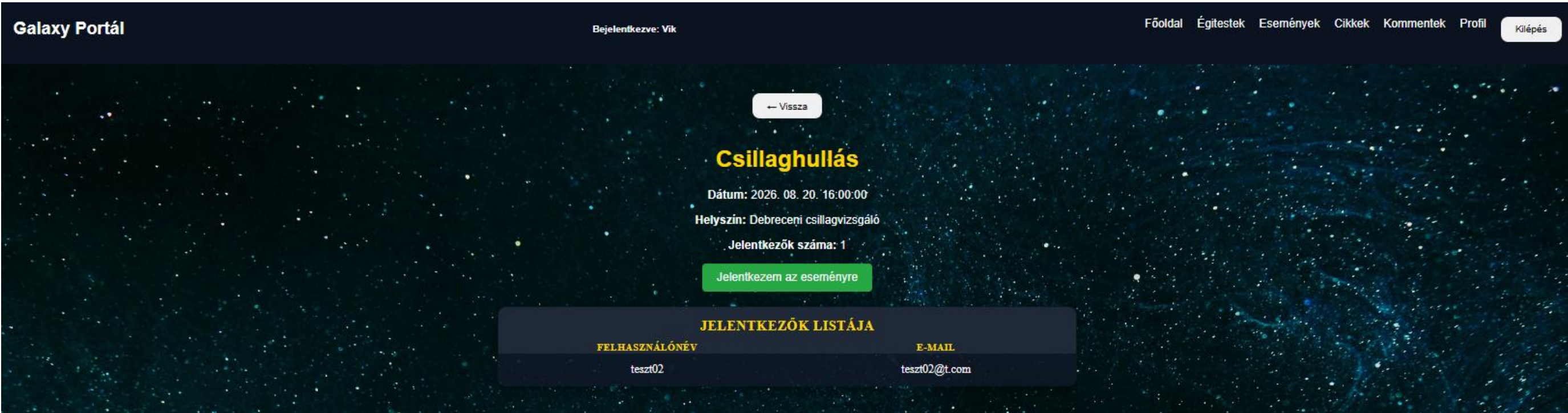
Esemény részleteinek megtekintése:

1. Kattints egy esemény "Részletek" gombjára
2. A részletes oldalon megjelenik:
 - Az esemény címe, időpontja, helyszíne
 - Az esemény részletes leírása
 - A jelentkezők száma
 - A jelentkezők listája



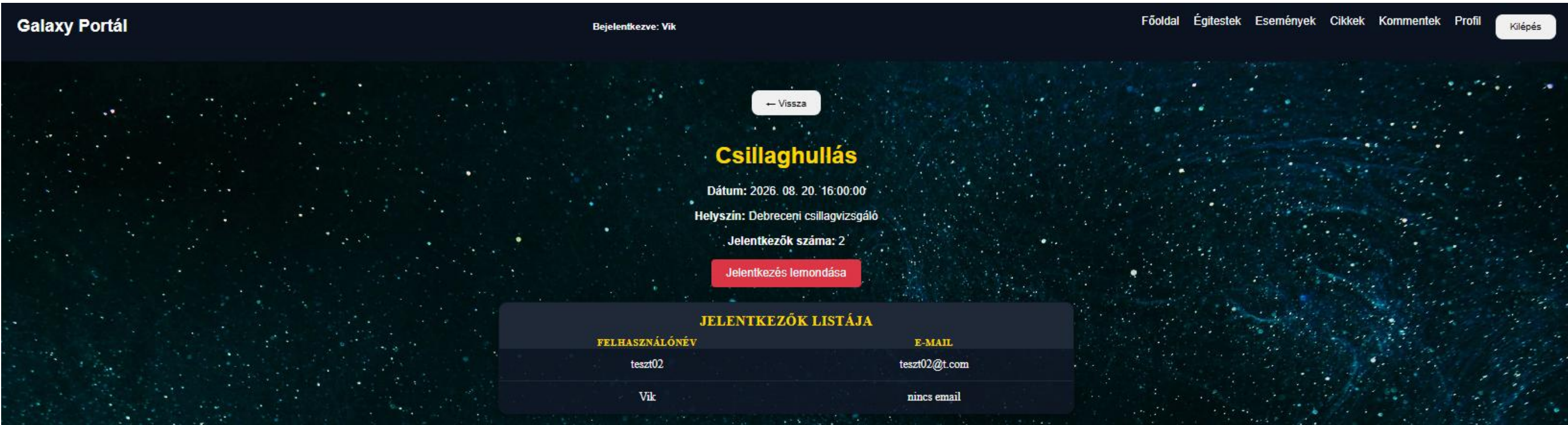
Eseményre jelentkezés:

- 1. Az esemény részletes oldalán kattints a "Jelentkezem az eseményre" gombra
- 2. A gomb felirata "Jelentkezés lemondása" -ra változik
- 3. A jelentkezők listája frissül az új jelentkezővel



Jelentkezés lemondása:

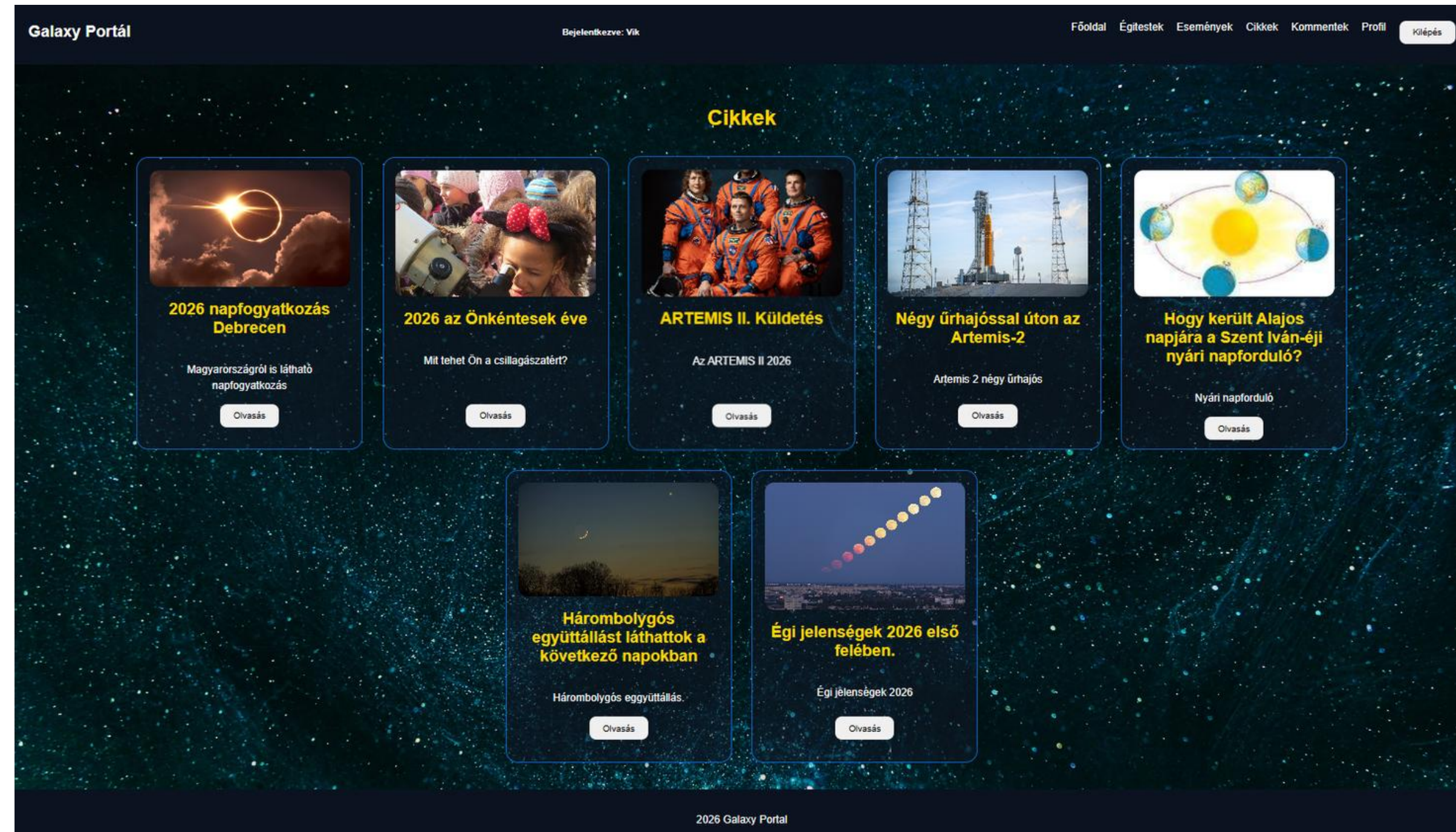
- 1. Az esemény részletes oldalán kattints a "Jelentkezés lemondása" gombra
- 2. A gomb felirata "Jelentkezem az eseményre" -ra változik



A Cikkek oldalon a csillagászati cikkek listája jelenik meg kártyás nézetben.

Cikkek böngészése:

1. Kattints a navigációs sáv "Cikkek" menüpontjára
2. Minden kártyán látható:
 - A cikk címe
 - A cikk rövid leírása
 - A cikk képe



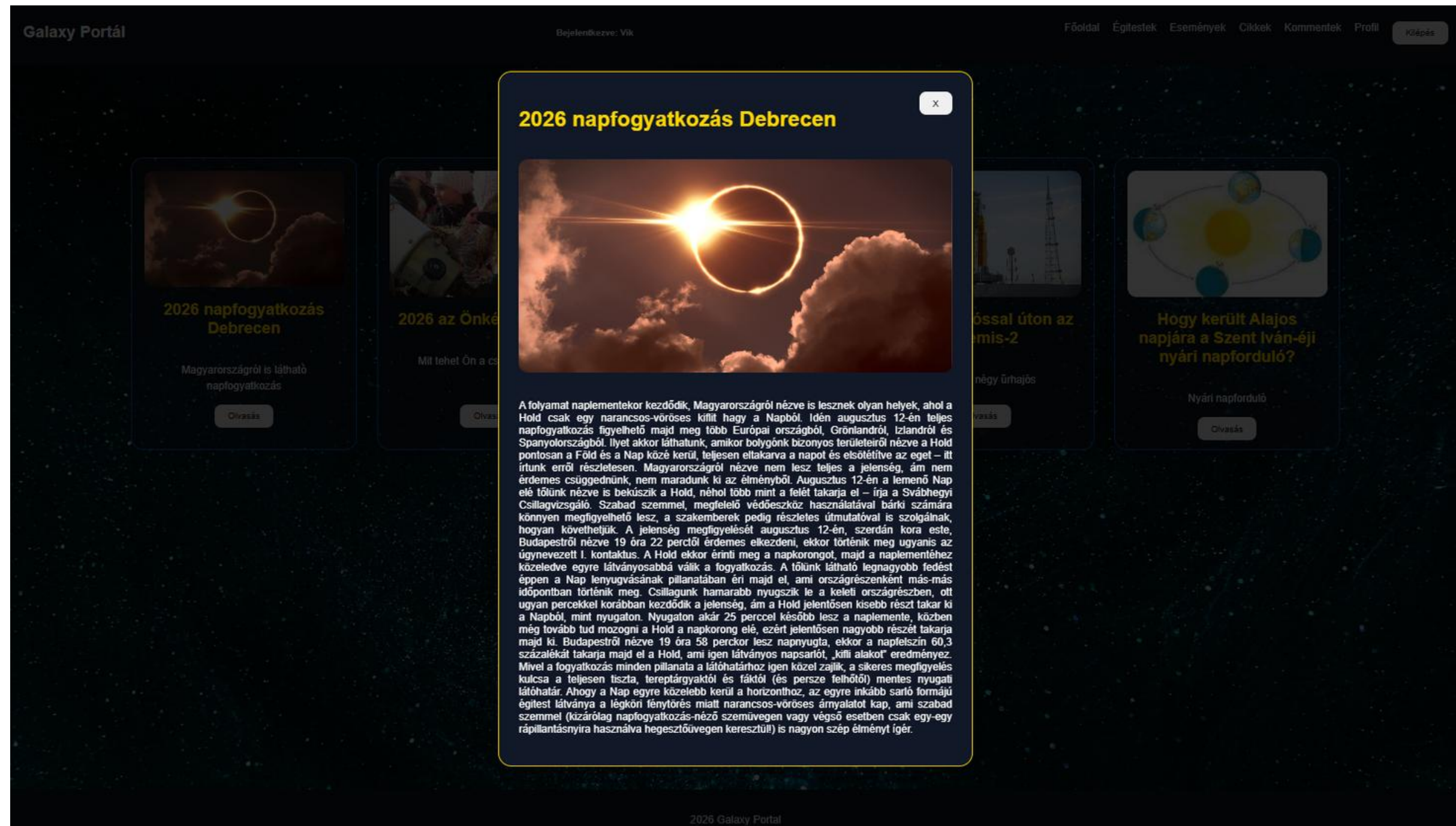
Teljes cikk megtekintése:

1. Kattints egy cikk "Olvasás" gombjára

2. Egy modál ablak nyílik, amely tartalmazza:

- A cikk címét
- A cikk képét (ha van)
- A cikk videóját (ha van)
- A cikk teljes szövegét

3. A modál bezárásához kattints az "X" gombra



3.3.6. Kommentek

A Kommentek oldal csak bejelentkezett felhasználók számára érhető el. Itt lehet olvasni és írni kommenteket.

Kommentek olvasása:

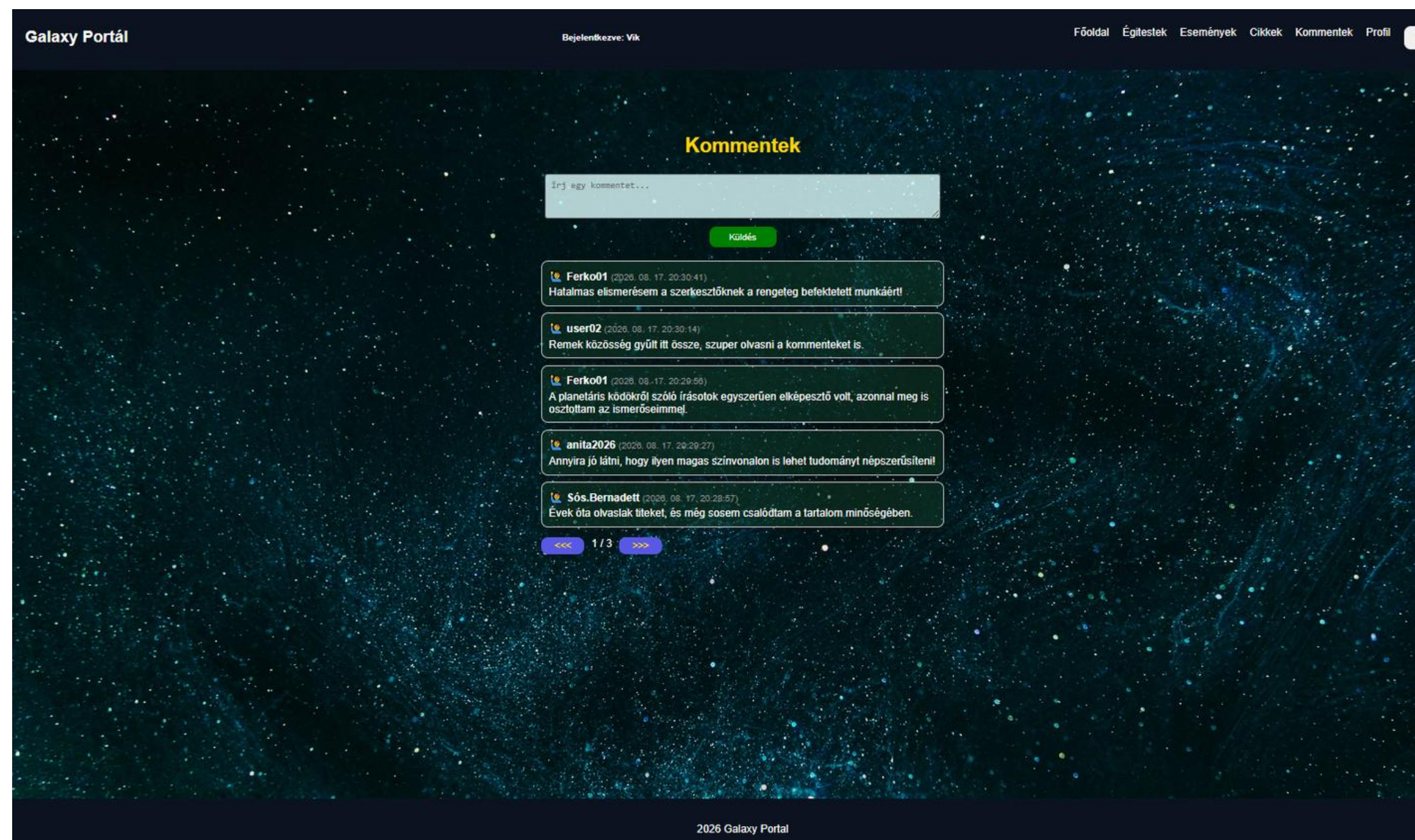
1. Kattints a navigációs sáv "Kommentek" menüpontjára
2. A kommentek listája megjelenik

Új komment írása:

1. Írd be a komment szövegét a szövegmezőbe
2. Kattints a "Küldés" gombra
3. A komment megjelenik a lista elején

Kommentek lapozása:

- A kommentek oldalanként 5 elemmel jelennek meg
- Használd az "Előző" és "Következő" gombokat a lapozáshoz



A Profil oldalon a bejelentkezett felhasználó adatai jelennek meg.

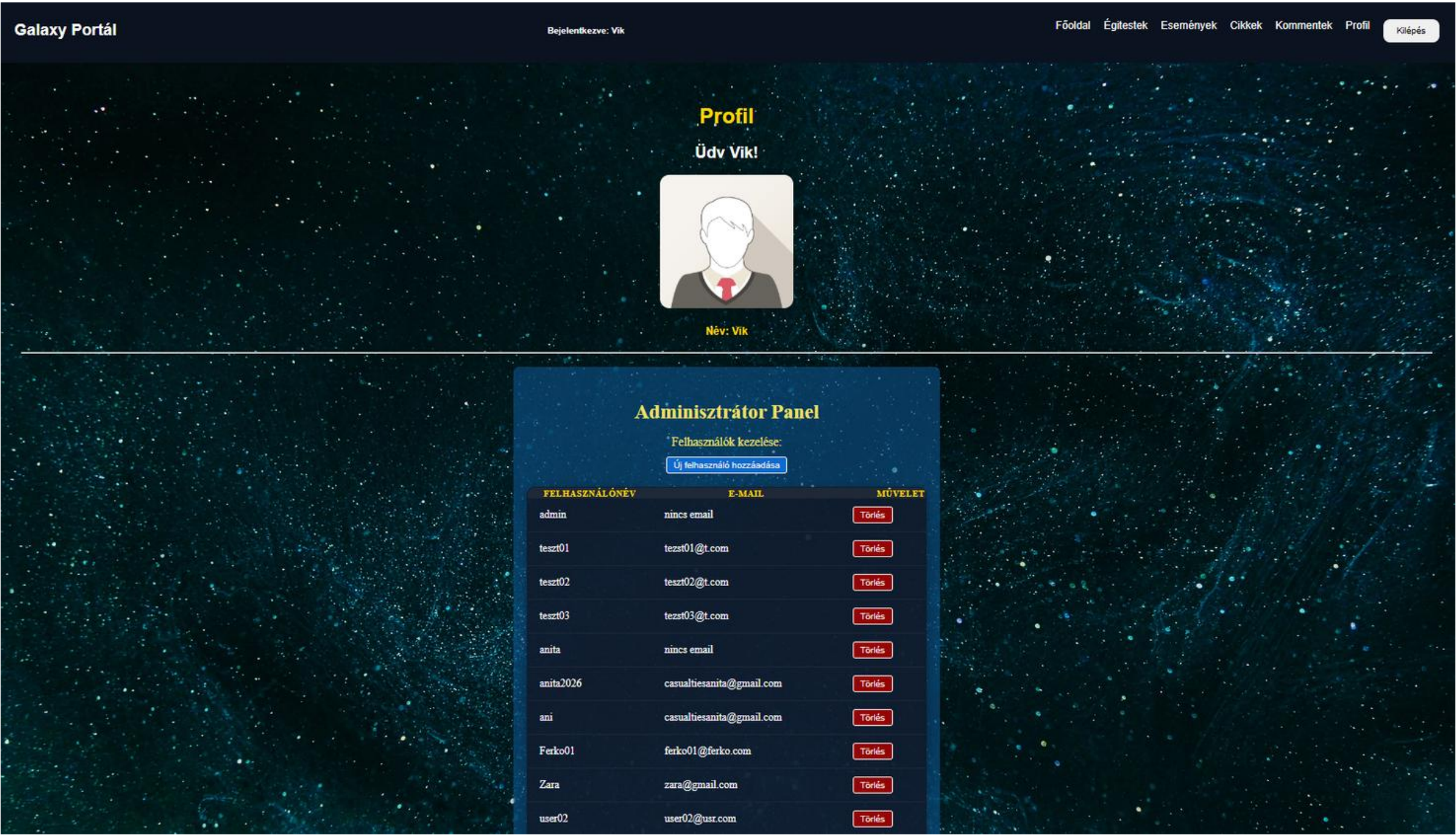
Profil megtekintése:

- 1. Kattints a navigációs sáv "Profil" menüpontjára
- 2. Megjelenik a felhasználóneved és egy profilképed

Adminisztrátori funkciók:

Ha adminisztrátori jogosultsággal rendelkezel (superuser), a Profil oldalon megjelenik egy "Adminisztrátori felület" panel:

- 1. Felhasználók listája: Az összes regisztrált felhasználó listája
- 2. Felhasználó törlése:
 - Kattints a "Törlés" gombra a felhasználó mellett
 - Erősítsd meg a törlést a felugró ablakban



3.3.8. Kijelentkezés

A kijelentkezés a navigációs sáv jobb oldalán található "Kilépés" gombbal történik.

4.1. Fejlesztői környezet és technológiák

A Galaxy Portal egy modern webalkalmazás, amely a legújabb iparági standardoknak megfelelően készült. A következő technológiai stack-et alkalmaztam a fejlesztés során:

4.1.1. Frontend technológiák

React

A React egy nyílt forráskódú JavaScript keretrendszer felhasználói felületek építésére. A React komponensalapú architektúrája lehetővé tette a kód újrahasznosíthatóságát és a felület moduláris felépítését. A virtuális DOM használata biztosítja a gyors és reszponzív felhasználói élményt.

A React választását az alábbi tulajdonságai indokolták:

- **Komponensalapúság:** A felület kisebb, újrahasznosítható komponensekre bontható
- **Virtuális DOM:** Gyors és hatékony megjelenítés
- **Hook-ok:** useState, useEffect, useContext a hatékony állapotkezeléshez
- **Nagy közösség:** Kiterjedt dokumentáció és támogatás

Vite

A Vite egy modern build eszköz és fejlesztői szerver. Előnyei:

- **Gyors indulás:** Natív ES modulok használata a böngészőben
- **Gyors HMR:** A kód módosításai szinte azonnal megjelennek
- **Optimális build:** Rollup használata éles környezetben

React Router DOM

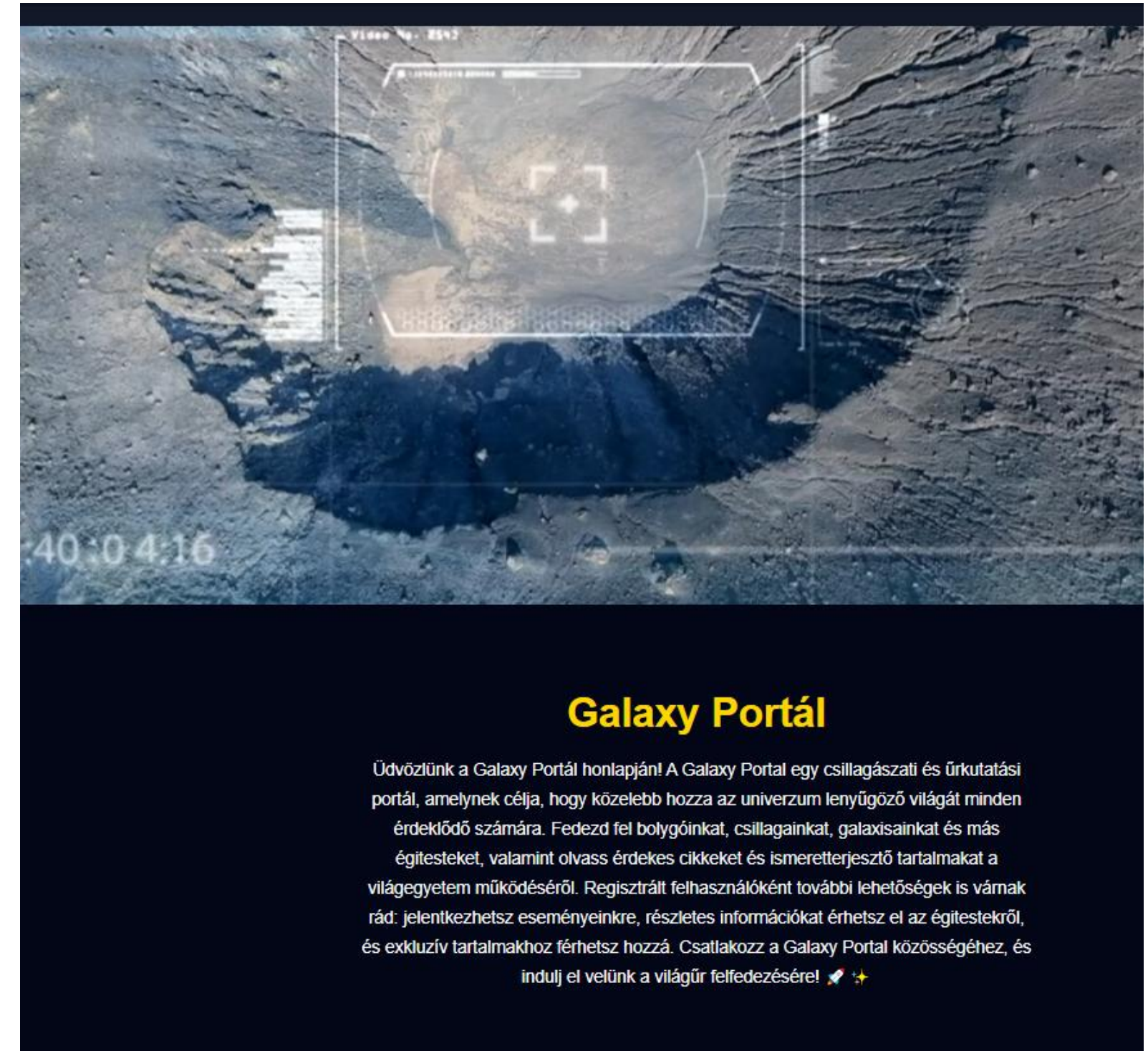
A React Router DOM biztosítja az alkalmazás navigációját. Lehetővé teszi az URL-ek és a megjelenítendő komponensek összekapcsolását. Fő elemei:

- BrowserRouter – a router komponens
- Routes és Route – útvonalak definiálása
- Navigate – programozott navigáció
- useNavigate, useParams – hook-ok a navigációhoz

Axios

Az Axios egy HTTP kliens könyvtár, amely egyszerűvé teszi a HTTP kérések küldését. A projektben az Axios a következő célokra szolgál:

- API hívások indítása
- Request interceptor: Authorization fejléc hozzáadása minden kéréshez
- Response interceptor: 401-es hibák kezelése, token frissítés



jwt-decode

A jwt-decode könyvtár a JWT token tartalmának kliensoldali kiolvasására szolgál. Lehetővé teszi a felhasználói adatok (felhasználónév, jogosultságok) kinyerését a tokenből.

JWT

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTEyMzQ1MDIyfQ.JXkWeyFuJJ158LBncFEaH-0x6mCUur8jySrno-1qFS0
```

Copy JWT

HeaderHS256

{
 "alg": "HS256",
 "typ": "JWT"
}

Payload

{
 "sub": "1234567890",
 "name": "John Doe",
 "iat": 1516239022
}

Signing Key

HMACSHA256(
 base64UrlEncode(header) + "." +
 base64UrlEncode(payload),

---Enter-your-256-bit-key---

)

⊗ Signature Invalid!

Note: We do not store any information in our database and all processing is done on the client side.

A backend Django REST API-ként szolgál, amely biztosítja az adatokat és a hitelesítési szolgáltatásokat.

Python:

A nyelv tiszta szintaxisa és hatalmas ökoszisztémája felgyorsította a fejlesztést. Kézenfekvő volt terjedelmes dokumentációja és támogatottsága miatt is.

Django:

A Django keretrendszerre esett a választás a "batteries included" tulajdonsága miatt. Segítségével egyszerűen lehet professzionális admin felületet, hitelesítési rendszert vagy akár URL-kezelést megoldani. Beépített ORM-rendszere lehetővé tette, hogy az adattáblákat Python osztályokként kezeljük.

Django REST Framework:

A DRF használatával szabványos, jól dokumentált és könnyen tesztelhető RESTful API végpontokat alakíthattam ki.

Simple JWT:

Ez a könyvtár biztosítja az állapotmentes hitelesítést. A tokenek használata egyszerűbbé teszi a kliens és a szerver közötti biztonságos kommunikációt.

4.1.3. Adatbázis

Az adatbázis-motor kiválasztásakor az SQLite mellett döntöttem, mivel a Django ORM-en keresztül az adatbázis-kezelés motorfüggetlen. Fájlalapú felépítése miatt nem igényel külön szerverüzemeltetést, ami ideálissá tette a gyors fejlesztéshez és a hordozhatósághoz.

4.1.4. Verziókezelés

A Git lehetővé tette a forráskód verzióinak követését, a különböző fejlesztési fázisok elkülönítését és a biztonságos visszaállítási pontok létrehozását. A GitHub platform biztosította a kód tárolását.

4.2. Adatbázis modell

Az adatbázis modell a Django beépített User modelljére épül. A backend az alábbi főbb entitásokat kezeli:

4.2.1. Felhasználók (User)

A Django beépített User modellje biztosítja a felhasználókezelést. Tartalmazza a felhasználónevet, e-mail címet, jelszót és a jogosultsági adatokat (is_staff, is_superuser). A frontend a JWT tokenből dekódolva éri el ezeket az adatokat.

4.2.2. Égitestek (Egitest)

Az égitestek táblája tartalmazza az űrkutatási objektumok adatait:

Mező	Típus	Leírás
id	Integer	Elsődleges kulcs
egitestNev	CharField	Égitest neve
egitestDescription	TextField	Részletes leírás
egitestImage	CharField	Kép URL-je
egitestVideo	CharField	Videó URL-je

4.2.3. Események (Esemeny)

Az események táblája tartalmazza a csillagászati események adatait:

Mező	Típus	Leírás
id	Integer	Elsődleges kulcs
esemenyNev	CharField	Esemény neve
esemenyDatum	DateTimeField	Esemény dátuma és időpontja
esemenyHelyszin	CharField	Esemény helyszíne
esemenyLeiras	TextField	Részletes leírás
jelentkezők	ManyToManyField	Jelentkezett felhasználók

4.2.4. Cikkek (Cikk)

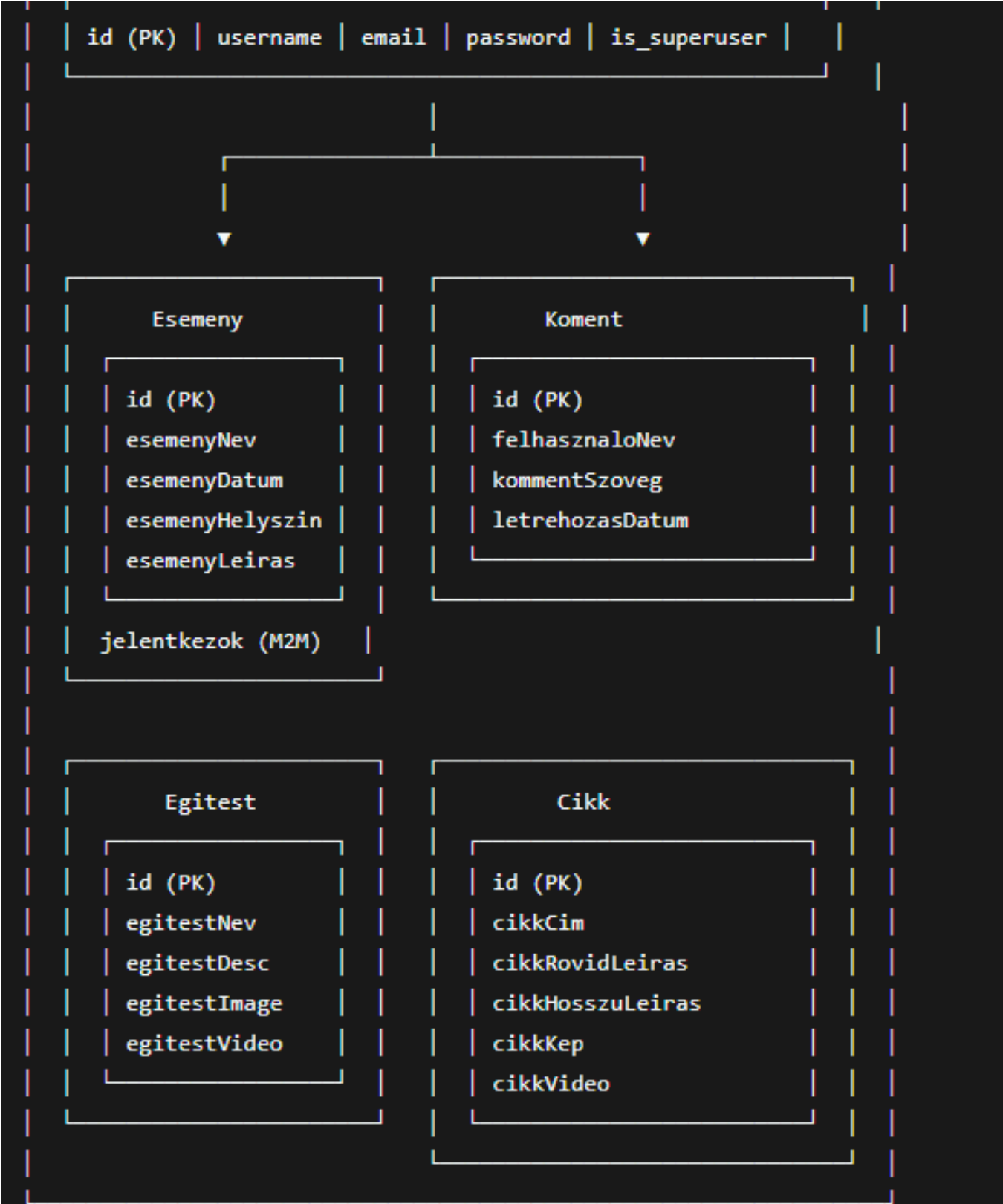
A cikkek táblája tartalmazza a csillagászati szakcikkek adatait:

Mező	Típus	Leírás
id	Integer	Elsődleges kulcs
cikkCim	CharField	Cikk címe
cikkRovidLeiras	TextField	Rövid leírás
cikkHosszuLeiras	TextField	Teljes tartalom
cikkKep	CharField	Kép URL-je
cikkVideo	CharField	Videó URL-je

4.2.5. Kommentek (Komment)

A kommentek táblája tartalmazza a felhasználók által írt kommenteket:

Mező	Típus	Leírás
id	Integer	Elsődleges kulcs
felhasznaloNev	CharField	Kommentelő neve
kommentSzoveg	TextField	Komment tartalma
letrehozasDatum	DateTimeField	Létrehozás dátuma



4.3. Kiemelt kódrészletek

A fejlesztés során több olyan kódrészlet is kiemelt szerepet kapott, amelyek az alkalmazás magját képezik.

4.3.1. AuthContext – Globális állapotkezelés

Az AuthContext.jsx biztosítja a bejelentkezett felhasználó globális állapotát. A Context API segítségével az alkalmazás bármely komponense elérheti a felhasználói adatokat anélkül, hogy prop-drilling-re lenne szükség.

Ez a kódrészlet azért fontos, mert:

- Biztosítja a felhasználói állapot perzisztens tárolását (oldalbetöltés után is megmarad)
- A JWT token dekódolásával kinyeri a felhasználói adatokat
- Egységes login/logout felületet biztosít az egész alkalmazás számára

```
import { createContext, useState, useContext, useEffect } from 'react';
import { jwtDecode } from 'jwt-decode';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);

  // Felhasználó visszaállítása oldalbetöltéskor
  useEffect(() => {
    const storedUser = localStorage.getItem('user');
    if (storedUser) {
      setUser(JSON.parse(storedUser));
    }
  }, []);

  // Bejelentkezés - token dekódolása és állapot mentése
  const login = (tokenOrUser) => {
    let userData;
    if (typeof tokenOrUser === 'string') {
      const decoded = jwtDecode(tokenOrUser);
      userData = {
        username: decoded.username,
        id: decoded.user_id,
        is_staff: decoded.is_staff,
        is_superuser: decoded.is_superuser,
      };
    } else {
      userData = tokenOrUser;
    }
    setUser(userData);
    localStorage.setItem('user', JSON.stringify(userData));
  };

  // Kijelentkezés - állapot és localStorage törlése
  const logout = () => {
    setUser(null);
    localStorage.removeItem('user');
    localStorage.removeItem('access_token');
    localStorage.removeItem('refresh_token');
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

4.3.2. ProtectedRoute – Védett útvonalak

A ProtectedRoute komponens feladata, hogy ellenőrizze a felhasználó bejelentkezési állapotát, és ennek megfelelően vagy megjelenítse a kért tartalmat, vagy átirányítsa a felhasználót a bejelentkezési oldalra.

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

function ProtectedRoute({ children }) {
  const { user } = useAuth();

  if (!user) {
    return <Navigate to="/login" replace />;
  }

  return children;
}

export default ProtectedRoute;
```

A kódrészlet jelentősége:

- Egyszerű, újrahasznosítható komponens
- Minden védett útvonalnál ugyanazt a logikát alkalmazza
- A /login oldalra irányítja a nem bejelentkezett felhasználókat

4.3.3. Axios interceptor – Token frissítés

Az axiosInstance.js fájlban található interceptor kezeli a token frissítést. 401-es válasz esetén megpróbálja a refresh token segítségével új access tokenet szerezni.

```
import axios from 'axios';

const BACKEND_URL = 'http://127.0.0.1:8000';

const axiosInstance = axios.create({
  baseURL: BACKEND_URL,
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor - Authorization fejléc hozzáadása
axiosInstance.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('access_token');
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

// Response interceptor - Token frissítés 401-es hibánál
axiosInstance.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

    if (error.response?.status === 401 && !originalRequest._retry) {
```

```
      if (error.response?.status === 401 && !originalRequest._retry) {
        originalRequest._retry = true;

        try {
          const refreshToken = localStorage.getItem('refresh_token');
          const response = await axios.post(`${BACKEND_URL}/api/auth/refresh/`, {
            refresh: refreshToken,
          });

          const newAccessToken = response.data.access;
          localStorage.setItem('access_token', newAccessToken);

          originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
          return axiosInstance(originalRequest);
        } catch (refreshError) {
          // Refresh token sikertelen - kijelentkeztetés
          localStorage.removeItem('access_token');
          localStorage.removeItem('refresh_token');
          localStorage.removeItem('user');
          window.location.href = '/login';
          return Promise.reject(refreshError);
        }
      }

      return Promise.reject(error);
    }
  },
  (error) => Promise.reject(error)
);

export default axiosInstance;
```

Ez a kódrészlet azért kiemelten fontos, mert:

- Automatikusan kezeli a lejárt access token esetét
- A felhasználó számára észrevétlenül frissíti a token
- Sikertelen refresh esetén biztonságosan kijelentkeztet

4.3.4. API adatátalakítás – Adapter minta

Az api.js fájlban az adatátalakítás (adapter minta) segítségével a backend magyar mezőneveit a frontend egységes nevekre alakítja.

A kódrészlet jelentősége:

- A frontend komponensek nem függenek a backend adatstruktúrájától
- Ha a backend változik, elegendő az API réteget módosítani
- A média URL-ek kezelése egységes (relatív URL-ek kiegészítése a backend címmel)

4.3.5. Kommentek lapozása

A kommentek oldalon kliensoldali lapozás valósult meg, oldalanként 5 elemmel.

Ez a kódrészlet azért fontos, mert:

- Kis adatmennyiségnél egyszerű és jól követhető megoldás
- Javítja a felhasználói élményt nagyobb kommentmennyiség esetén
- Könnyen cserélhető szerveroldali lapozásra a jövőben

4.4. Tesztelés és hibajavítás

A fejlesztés során nagy hangsúlyt fektettem a folyamatos tesztelésre és a hibák gyors javítására. A tesztelés célja az volt, hogy az alkalmazás stabilan, hibamentesen és felhasználóbarát módon működjön.

4.4.1. Tesztelési módszerek

Manuális funkcionális tesztelés

A rendszer minden modulját lépésről lépésre ellenőriztem:

- Útvonalak működése (navigáció)
- Védett oldalak átirányítása
- Be- és kijelentkezés
- LocalStorage tartalma
- API kérések státuszkódja
- Mobil nézet

A frontend működését több böngészőben is ellenőriztem:

- Google Chrome (elsődleges)
- Microsoft Edge
- Mozilla Firefox
- Safari

Reszponzivitás tesztelése

A felületet különböző képernyőméreteken is teszteltem:

- Mobil (320px - 480px)
- Tablet (768px - 1024px)
- Laptop (1024px - 1440px)
- Nagy felbontású monitor (1920px+)

Backend API tesztelése

A Django REST API végpontokat Postman és böngésző segítségével ellenőriztem:

- Helyes válaszok
- Hibakezelés
- Jogosultságok
- Adatvalidáció

4.4.2. Előkerült hibák és javításuk

A fejlesztés során több kisebb-nagyobb hiba került felszínre, amelyeket dokumentáltam és javítottam. Az alábbiakban a legfontosabbak szerepelnek.

➤ Hibás token kezelés oldalbetöltéskor

Hiba: Oldalbetöltéskor a felhasználói állapot elveszett, még akkor is, ha volt érvényes token a localStorage-ban.

Ok: A useEffect nem ellenőrizte a token érvényességét, csak a user objektumot olvasta vissza a localStorage-ból.

Megoldás: A AuthProvider komponens useEffect hook-jában hozzáadtam a token ellenőrzését és a dekódolt adatok alapján a felhasználói állapot beállítását.

```
useEffect(() => {  
  const token = localStorage.getItem('access_token');  
  const storedUser = localStorage.getItem('user');  
  
  if (token && storedUser) {  
    try {  
      const decoded = jwtDecode(token);  
      const userData = JSON.parse(storedUser);  
      // Ellenőrizzük, hogy a token-ben lévő adatok megegyeznek-e  
      if (decoded.username === userData.username) {  
        setUser(userData);  
      } else {  
        // Ha nem, töröljük a hibás adatokat  
        localStorage.removeItem('user');  
      }  
    } catch (error) {  
      // Hibás token esetén törlés  
      localStorage.removeItem('access_token');  
      localStorage.removeItem('user');  
    }  
  }  
}, []);
```

Hiba: A backendről érkező ékezetes szövegek helytelenül jelentek meg a frontenden.

Ok: A karakterkódolás nem volt egységes a backend és frontend között.

Megoldás: A backend válaszfejlécében beállítottam a megfelelő karakterkódolást, a frontenden pedig az Axios konfigurációban biztosítottam az UTF-8 használatát.

```
// axiosInstance.js
const axiosInstance = axios.create({
  baseURL: BACKEND_URL,
  timeout: 10000,
  headers: {
    'Content-Type': 'application/json; charset=utf-8',
    'Accept': 'application/json; charset=utf-8',
  },
});
```

Hiba: Eseményre jelentkezés után a gomb felirata nem változott, és a jelentkezők száma sem frissült.

Ok: A frontend a szerver válasza alapján frissítette az állapotot, de a komponens nem renderelődött újra megfelelően.

Megoldás: A handleRegistration függvényben a sikeres API hívás után újra betöltöttem az esemény adatait, és frissítettem a state-et.

```
const handleRegistration = async () => {  
  try {  
    setLoading(true);  
    await toggleEventRegistration(id);  
    // Esemény újratöltése a friss adatokért  
    const updatedEvent = await getEventById(id);  
    setEvent(updatedEvent);  
    setIsRegistered(updatedEvent.isRegistered);  
    setAttendees(updatedEvent.attendees);  
  } catch (error) {  
    setError('Hiba történt a jelentkezés során.');  } finally {  
    setLoading(false);  
  }  
};
```


Média URL-ek hiányosak voltak

Hiba: Az égitestek és cikkek képei, videói nem jelentek meg, mert relatív URL-eket kaptam a backendtől.

Ok: A backend relatív elérési utakat adott vissza a médiafájlokhoz.

Megoldás: Az api.js fájlban az adatátalakításkor ellenőriztem, hogy az URL teljes-e, és ha nem, hozzáfűztem a backend címét.

```
image: planet.egitestImage ?  
  (planet.egitestImage.startsWith('http') ?  
    planet.egitestImage :  
    `${BACKEND_URL}${planet.egitestImage}`) :  
  null
```

A Galaxy Portal fejlesztése során egy olyan modern webalkalmazást hoztam létre, amely választ ad a csillagászati és űrkutatási tartalmak iránti érdeklődésre. A projekt bebizonyította, hogy a Django robusztus backendje és a React dinamikus frontendje közötti szimbiózis lehetővé teszi skálázható és biztonságos rendszerek építését.

Elért eredmények:

- Teljes funkcionalitású webes portál létrehozása
- Modern technológiák használata (React, Vite, Axios, JWT)
- JWT alapú hitelesítés token frissítéssel
- Reszponzív, mobilbarát felület
- Karbantartható, moduláris kódszerkezet
- Intuitív felhasználói élmény

Tapasztalatok és tanulságok:

- JWT kezelés: A token alapú hitelesítés és automatikus frissítés megvalósítása kihívást jelentett, de a végeredmény egy biztonságos és felhasználóbarát rendszer.
- API kommunikáció: Az Axios interceptorok használata jelentősen megkönnyítette a token kezelést és a hibakezelést.
- Állapotkezelés: A Context API elegendőnek bizonyult a globális felhasználói állapot kezelésére, nem volt szükség komplexebb megoldásokra (pl. Redux).
- Reszponzivitás: A mobilbarát kialakítás elengedhetetlen a modern webalkalmazásoknál.

A projekt sikeresen teljesítette a kitűzött célokat: egy átlátható, könnyen kezelhető és technikailag stabil rendszert alkottam meg. A Galaxy Portal készen áll a jövőbeni bővítésekre és továbbfejlesztésekre.

6. TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK

A Galaxy Portal jelenlegi funkcionalitása stabil és jól használható, ugyanakkor több olyan fejlesztési irány is azonosítható, amelyek tovább növelhetik a hatékonyságot és a felhasználói élményt.

6.1. Felhasználói élmény fejlesztése

- Toast értesítések: A felhasználói műveletek (pl. bejelentkezés, eseményre jelentkezés, komment írása) eredményének vizuális jelzése
- Betöltési állapotok: Loading spinner és skeleton screen-ek használata az adatok betöltése közben
- Dark/Light mód: Felhasználói preferencia alapján választható téma

6.2. Adatkezelés fejlesztése

- Szerveroldali lapozás: Nagy adathalmazok (pl. kommentek) esetén szerveroldali lapozás implementálása
- Keresés és szűrés: Égitestek, események és cikkek szűrésének és keresésének lehetősége
- Részletes statisztikák: Felhasználói aktivitás, események népszerűsége

6.3. Biztonsági fejlesztések

- Környezeti változók: Az API alapcím és más érzékeny adatok környezeti változókba szervezése
- Token biztonság: A localStorage helyett biztonságosabb tárolási megoldások (pl. HttpOnly cookie-k) mérlegelése

6.4. Funkcionális bővítések

- Értesítési rendszer: Események közeledtéről, új kommentekről szóló értesítések
- Admin felület bővítése: Tartalmak (égitestek, események, cikkek) kezelésének lehetősége admin felületen
- Többnyelvűség: Angol nyelvi támogatás bevezetése
- PWA támogatás: Offline használat, telepíthetőség

6.5. Technikai fejlesztések

- Egységtesztek: React komponensek tesztelése Jest és React Testing Library segítségével
- CI/CD pipeline: Automatikus tesztelés és deploy beállítása
- Dockerizálás: Konténerizált fejlesztői és éles környezet

7. IRODALOMJEGYZÉK

Az alkalmazás fejlesztése során az alábbi források kerültek felhasználásra:

React

- Hivatalos dokumentáció: <https://react.dev/>
- React útmutató: <https://react.dev/learn>

React Router

- Hivatalos dokumentáció: <https://reactrouter.com/>

Vite

- Hivatalos dokumentáció: <https://vitejs.dev/>

Axios

- Hivatalos dokumentáció: <https://axios-http.com/>
-

JWT

- Hivatalos dokumentáció: <https://jwt.io/>
- jwt-decode: <https://www.npmjs.com/package/jwt-decode>
-

Django

- Hivatalos dokumentáció: <https://www.djangoproject.com/>
- Django REST Framework: <https://www.django-rest-framework.org/>

Python

- Hivatalos dokumentáció: <https://docs.python.org/3/>

Segédanyagok

- W3Schools: <https://www.w3schools.com/>
- MDN Web Docs: <https://developer.mozilla.org/>
- Stack Overflow: <https://stackoverflow.com/>

Dokumentáció Vége.

Köszönöm a figyelmet!!